



# K.R. Mangalam University School of Engineering & Technology

Fundamentals Of Java Programming Lab

(ENCA203) Assignment 2 Student Management System

Submitted by:

Name: Shivam Gaur

Roll no. 2401201016

Class: BCA (h) (AI & DS)

Submitted to:

Dr. Manish Kumar

GitHub Repository:

[https://github.com/shivamgaur1255-sketch/java\\_assignments-](https://github.com/shivamgaur1255-sketch/java_assignments-)

# Codes:

```
J Calculator.java 1 J StudentManagementSystem.java 1 X StudentApp
J StudentManagementSystem.java > StudentManagementSystem > main(String[])
1 import java.util.*;
2
3 // Abstract Class Person
4 abstract class Person {
5     protected String name;
6     protected String email;
7
8     public Person(String name, String email) {
9         this.name = name;
10        this.email = email;
11    }
12
13    public abstract void displayInfo();
14 }
15
16 // Final Student class extending Person
17 final class Student extends Person {
18     private int rollNo;
19     private String course;
20     private double marks;
21     private String grade;
22
23     public Student(int rollNo, String name, String email, String course, double marks, String grade) {
24         super(name, email);
25         this.rollNo = rollNo;
26         this.course = course;
27         this.marks = marks;
28         this.grade = grade;
29     }
30
31     // Method Overriding
32     @Override
33     public void displayInfo() {
34         System.out.println("Student Info: ");
35         System.out.println("Roll No: " + rollNo);
36         System.out.println("Name: " + name);
37         System.out.println("Email: " + email);
38         System.out.println("Course: " + course);
39         System.out.println("Marks: " + marks);
40         System.out.println("Grade: " + grade);
41         System.out.println("-----");
42     }
43
44     // Method Overloading
45     public void displayInfo(String researchArea) {
46
47         // Method Overloading
48         public void displayInfo(String researchArea) {
49             System.out.println("Student Info: ");
50             System.out.println("Roll No: " + rollNo);
51             System.out.println("Name: " + name);
52             System.out.println("Email: " + email);
53             System.out.println("Course: " + course);
54             System.out.println("Research Area: " + researchArea);
55             System.out.println("Marks: " + marks);
56             System.out.println("Grade: " + grade);
57             System.out.println("-----");
58         }
59
60         // Final method demonstration
61         public final void finalDisplayInfo() {
62             displayInfo();
63             System.out.println("This is a final method in a final class.");
64         }
65
66         // Getters/Setters
67         public int getRollNo() { return rollNo; }
68         public void setMarks(double marks) { this.marks = marks; }
69         public void setGrade(String grade) { this.grade = grade; }
70         public String getCourse() { return course; }
71         public double getMarks() { return marks; }
72         public String getGrade() { return grade; }
73         public String getName() { return name; }
74         public String getEmail() { return email; }
75
76         @Override
77         protected void finalize() throws Throwable {
78             System.out.println("Finalize method called before object is garbage collected.");
79             super.finalize();
80         }
81     }
82
83     // Interface for RecordActions
84     interface RecordActions {
85         boolean addStudent(Student student);
86         boolean deleteStudent(int rollNo);
87         boolean updateStudent(int rollNo, Student updatedStudent);
88         Student searchStudent(int rollNo);
89         List<Student> viewAllStudents();
90         void sortStudentsByMarks();
91     }
92 }
```

```
J Calculator.java 1 J StudentManagementSystem.java 1 x StudentApp
J StudentManagementSystem.java > StudentManagementSystem > main(String[])
81 interface RecordActions {
86     List<Student> viewAllStudents();
87     void sortStudentsByMarks();
88 }
89
90 // StudentManager class implementing RecordActions
91 class StudentManager implements RecordActions {
92     private Map<Integer, Student> studentMap;
93
94     public StudentManager() {
95         studentMap = new HashMap<>();
96     }
97
98     @Override
99     public boolean addStudent(Student student) {
100         if (studentMap.containsKey(student.getRollNo())) {
101             System.out.println("Duplicate roll number: " + student.getRollNo());
102             return false;
103         }
104         studentMap.put(student.getRollNo(), student);
105         return true;
106     }
107
108     @Override
109     public boolean deleteStudent(int rollNo) {
110         return studentMap.remove(rollNo) != null;
111     }
112
113     @Override
114     public boolean updateStudent(int rollNo, Student updatedStudent) {
115         if (!studentMap.containsKey(rollNo)) return false;
116         studentMap.put(rollNo, updatedStudent);
117         return true;
118     }
119
120     @Override
121     public Student searchStudent(int rollNo) {
122         return studentMap.get(rollNo);
123     }
124
125     @Override
126     public List<Student> viewAllStudents() {
127         return new ArrayList<>(studentMap.values());
128     }
129 }
```

```
J Calculator.java 1 J StudentManagementSystem.java 1 x StudentApp
J StudentManagementSystem.java > StudentManagementSystem > main(String[])
91 class StudentManager implements RecordActions {
130
131     @Override
132     public void sortStudentsByMarks() {
133         List<Student> students = viewAllStudents();
134         students.sort(Comparator.comparingDouble(Student::getMarks).reversed());
135         System.out.println(x: "--- Sorted Students by Marks ---");
136         for (Student s : students) {
137             s.displayInfo();
138         }
139     }
140
141 // Main class with menu-driven input
142 public class StudentManagementSystem {
143     public static void main(String[] args) {
144         StudentManager manager = new StudentManager();
145         Scanner sc = new Scanner(System.in);
146
147         while (true) {
148             System.out.println(x: "\n--- Student Management Menu ---");
149             System.out.println(x: "1. Add Student");
150             System.out.println(x: "2. Delete Student");
151             System.out.println(x: "3. Update Student");
152             System.out.println(x: "4. Search Student");
153             System.out.println(x: "5. View All Students");
154             System.out.println(x: "6. Sort Students by Marks");
155             System.out.println(x: "7. Exit");
156             System.out.print(s: "Enter choice: ");
157             int choice = sc.nextInt();
158             sc.nextLine(); // Consume newline
159
160             switch (choice) {
161                 case 1:
162                     System.out.print(s: "Enter rollNo: ");
163                     int rollNo = sc.nextInt();
164                     sc.nextLine();
165                     System.out.print(s: "Enter name: ");
166                     String name = sc.nextLine();
167                     System.out.print(s: "Enter email: ");
168                     String email = sc.nextLine();
169                     System.out.print(s: "Enter course: ");
170                     String course = sc.nextLine();
171                     System.out.print(s: "Enter marks: ");
172                     double marks = sc.nextDouble();
```

```
J Calculator.java 1 J StudentManagementSystem.java 1 X StudentApp
J StudentManagementSystem.java > StudentManagementSystem > main(String[])
142 public class StudentManagementSystem {
143     public static void main(String[] args) {
170         String course = sc.nextLine();
171         System.out.print(s:"Enter marks: ");
172         double marks = sc.nextDouble();
173         sc.nextLine();
174         System.out.print(s:"Enter grade: ");
175         String grade = sc.nextLine();
176
177         Student student = new Student(rollNo, name, email, course, marks, grade);
178         manager.addStudent(student);
179         break;
180
181     case 2:
182         System.out.print(s:"Enter rollNo to delete: ");
183         int delRoll = sc.nextInt();
184         manager.deleteStudent(delRoll);
185         break;
186
187     case 3:
188         System.out.print(s:"Enter rollNo to update: ");
189         int updRoll = sc.nextInt();
190         sc.nextLine();
191         System.out.print(s:"Enter updated name: ");
192         String updName = sc.nextLine();
193         System.out.print(s:"Enter updated email: ");
194         String updEmail = sc.nextLine();
195         System.out.print(s:"Enter updated course: ");
196         String updCourse = sc.nextLine();
197         System.out.print(s:"Enter updated marks: ");
198         double updMarks = sc.nextDouble();
199         sc.nextLine();
200         System.out.print(s:"Enter updated grade: ");
201         String updGrade = sc.nextLine();
202
203         Student updatedStudent = new Student(updRoll, updName, updEmail, updCourse, updMarks, updGrade);
204         manager.updateStudent(updRoll, updatedStudent);
205         break;
206
207     case 4:
208         System.out.print(s:"Enter rollNo to search: ");
209         int searchRoll = sc.nextInt();
210         Student found = manager.searchStudent(searchRoll);
211         if (found != null) {
212             found.displayInfo();
213
214             // Method overloading demonstration
215             System.out.print(s:"Enter research area if applicable (or leave blank): ");
216             sc.nextLine();
217             String researchArea = sc.nextLine();
218             if (!researchArea.isEmpty()) {
219                 found.displayInfo(researchArea);
220             }
221             else {
222                 System.out.println(x:"Student not found.");
223             }
224             break;
225
226     case 5:
227         List<Student> students = manager.viewAllStudents();
228         for (Student s : students) {
229             s.displayInfo();
230         }
231         break;
232
233     case 6:
234         manager.sortStudentsByMarks();
235         break;
236
237     case 7:
238         System.out.println(x:"Exiting...");
239         sc.close();
240         System.exit(status:0);
241
242     default:
243         System.out.println(x:"Invalid choice. Try again.");
244     }
245 }
246 }
247 }
```

# Output:

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS StudentApp

PS C:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding> c:: cd 'c:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding'; & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+Sho
wCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Shivam Gaur\AppData\Roaming\Code\User\workspaceStorage\b86f09e45bbf9afd5a31de64f223dfa7\redhat.java\jdt_ws\java_coding_d70f91f4\bin
\StudentManagementSystem'

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 1
Enter rollNo: 101
Enter name: Shivam
Enter email: shivam@mail.com
Enter course: BCA
Enter marks: 90
Enter grade: A

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 1
Enter rollNo: 102
Enter name: Ankita
Enter email: ankita@mail.com
Enter course: BSc
Enter marks: 95
Enter grade: A

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 1
Enter rollNo: 103
Enter name: Vyan
Enter email: vyan@mail.com
Enter course: BBA
Enter marks: 60

Enter course: BBA
Enter marks: 60
Enter grade: B

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 2
Enter rollNo to delete: 103

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 3
Enter rollNo to update: 103
Enter updated name: Vinayak
Enter updated email: vinayak@mail.com
Enter updated course: B.Tech
Enter updated marks: 80
Enter updated grade: B

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 4
Enter rollNo to search: 102
Student Info:
Roll No: 102
Name: Ankita
Email: ankita@mail.com
Course: BSc
Marks: 95.0
Grade: A
-----
Enter research area if applicable (or leave blank):
```

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 5
Student Info:
Roll No: 101
Name: Shivam
Email: shivam@mail.com
Course: BCA
Marks: 90.0
Grade: A
-----
Student Info:
Roll No: 102
Name: Ankita
Email: ankita@mail.com
Course: BSc
Marks: 95.0
Grade: A
-----

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 6
--- Sorted Students by Marks ---
Student Info:
Roll No: 102
Name: Ankita
Email: ankita@mail.com
Course: BSc
Marks: 95.0
Grade: A
-----
Student Info:
Roll No: 101
Name: Shivam
Email: shivam@mail.com
Course: BCA
Marks: 90.0
-----

Roll No: 101
Name: Shivam
Email: shivam@mail.com
Course: BCA
Marks: 90.0
Grade: A
-----

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
Course: BCA
Marks: 90.0
Grade: A
-----

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
Grade: A
-----

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student

--- Student Management Menu ---
1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. View All Students
6. Sort Students by Marks
5. View All Students
6. Sort Students by Marks
7. Exit
Enter choice: 7
Enter choice: 7
Exiting...
PS C:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding>
```

Explanation:

## Step-by-Step How It Works

- **When you start the program**, you see a menu with options like “Add Student,” “Delete Student,” “Update Student,” “Search Student,” “View All Students,” and “Sort Students by Marks.”
- **You choose an operation by entering a number (1–7).**
- For example, if you want to add a student:
  - The program asks you to enter details (roll number, name, email, course, marks, and grade).
  - It then saves those details as a new Student object.
- **StudentManager** is the part of the code that keeps track of all students, making sure roll numbers are unique so you cannot add the same student twice.
- You can:
  - **Delete** a student by entering their roll number.
  - **Update** a student’s details, again by roll number.
  - **Search** for a student and see their details.
  - **View all students** at once.
  - **Sort students by their marks** to easily see who performed best.
- The program keeps running until you pick “Exit.”
- Under the hood, it uses Java features like interfaces, abstract classes, and inheritance to organize the code neatly and make it flexible for changes